MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Dr. J.R.Arun kumar

Subject & Subject Code: Natural Language Processing (6CS5-13)

Semester: VI **Session:** 2023-24(Even Sem.)

Unit No.: IV **Date of Submission:** 23/02/2024

Sl.No.	Topics Covered	Page No.	Total pages
1	**Natural Language Generation:** Natural Language Generation (NLG) Architecture of NLG Systems	1-4	
2	Generation Tasks and Representations	5-7	
3	Application of NLG	8-11	
4	Machine Translation: Problems in Machine Translation	12-14	
5	Characteristics of Indian Languages	15-17	
6	Machine Translation Approaches	18-20	
7	Translation involving Indian Languages	21-23	

References Book:

Natural Language understanding by James Allen, Pearson Education 2008

NLP: A Paninian Perspective by Akshar Bharati, Vineet Chaitanya, and Rajeev Sangal, Prentice Hal

FACULTY HOD 23/3/24 DEAN ACADEMICS

UNIT 4

Natural Language Generation: Natural Language Generation (NLG):

LECTURE NO:01

WHAT IS NATURAL LANGUAGE GENERATION (NLG)?

Natural language generation (NLG) is the use of artificial intelligence (AI) programming to produce written or spoken narratives from a data set. NLG is related to human-to-machine and machine-to-human interaction, including computational linguistics, natural language processing (NLP) and natural language understanding (NLU).

Research about NLG often focuses on building computer programs that provide data points with context. Sophisticated NLG software can mine large quantities of numerical data, identify patterns and share that information in a way that is easy for humans to understand. The speed of NLG software is especially useful for producing news and other time-sensitive stories on the internet. At its best, NLG output can be published verbatim as web content.

HOW DOES NLG WORK?

NLG is a multi-stage process, with each step further refining the data being used to produce content with natural-sounding language. The six stages of NLG are as follows:

1. **Content analysis.** Data is filtered to determine what should be included in the content produced at the end of the process. This stage includes identifying the main topics in the source document and the relationships between them.

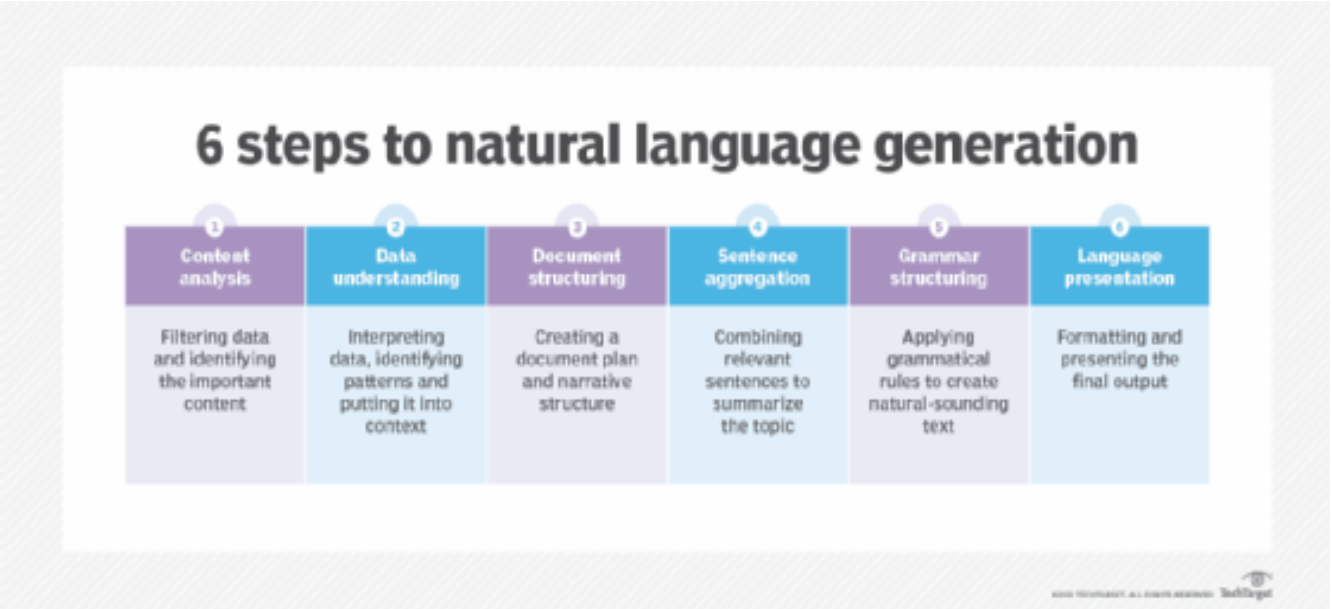
2. **Data understanding.** The data is interpreted, patterns are identified and it's put into context. Machine learning is often used at this stage.

3. **Document structuring.** A document plan is created and a narrative structure chosen based on the type of data being interpreted.

4. **Sentence aggregation.** Relevant sentences or parts of sentences are combined in ways that accurately summarize the topic.

5. **Grammatical structuring.** Grammatical rules are applied to generate natural-sounding text. The program deduces the syntactical structure of the sentence. It then uses this information to rewrite the sentence in a grammatically correct manner.

6. **Language presentation.** The final output is generated based on a template or format the user or programmer has selected.



HOW IS NLG USED?

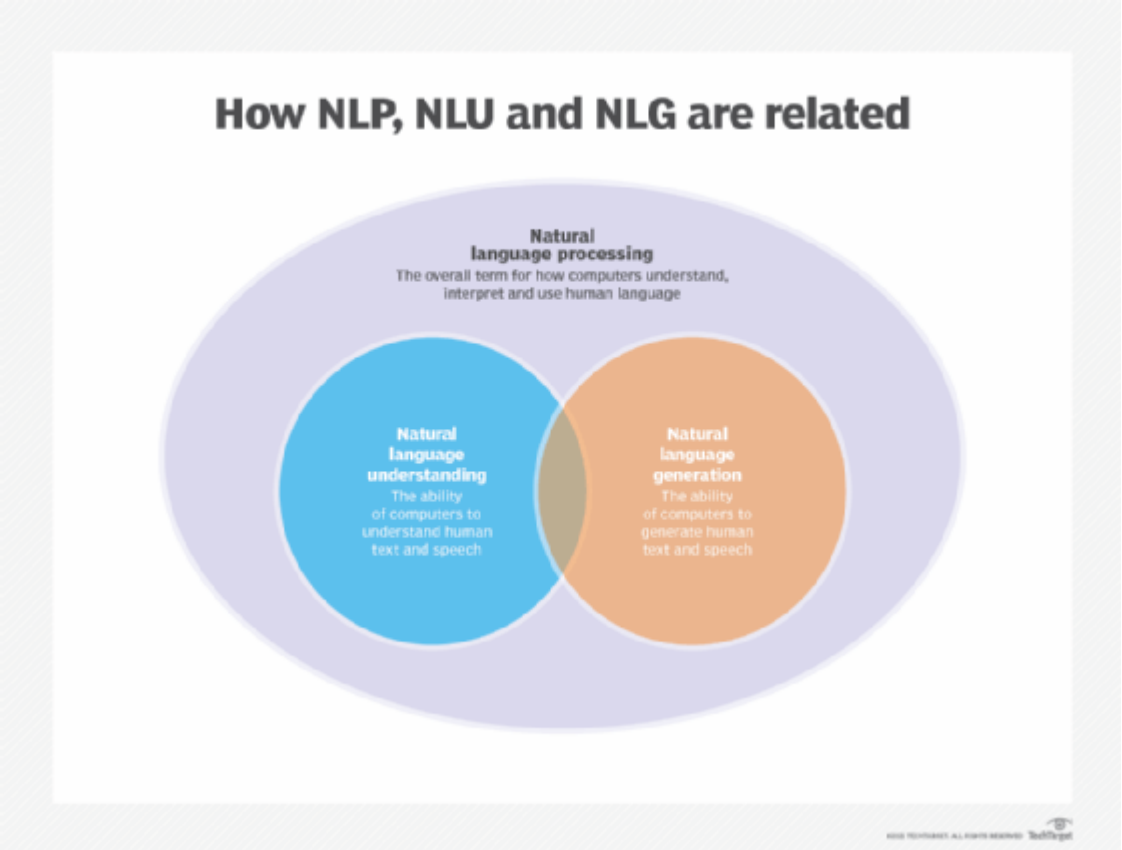
Natural language generation is being used in an array of ways. Some of the many uses include the following:

- generating the responses of chatbots and voice assistants such as Google's Alexa and Apple's Siri;
- converting financial reports and other types of business data into easily understood content for employees and customers;

- automating lead nurturing email, messaging and chat responses;
- personalizing responses to customer emails and messages;
- generating and personalizing scripts used by customer service representatives;
- aggregating and summarizing news reports;
- reporting on the status of internet of things devices; and
- creating product descriptions for e-commerce webpages and customer messaging.

NLG VS. NLU VS. NLP

NLP is an umbrella term that refers to the use of computers to understand human language in both written and verbal forms. NLP is built on a framework of rules and components, and it converts unstructured data into a structured data format.



UNIT 4

Natural Language Generation: Natural Language Generation (NLG):

LECTURE NO:02

ARCHITECTURE OF NLG SYSTEMS

STAGES OF NATURAL LANGUAGE GENERATION

- **Content determination:** Deciding the main content to be represented in a sentence or the information to mention in the text. For instance, in the pollen example above, deciding whether to explicitly mention that pollen level is 7 in the south-east.
- **Document structuring:** Deciding the structure or organization of the conveyed information. For example, deciding to describe the areas with high pollen levels first, instead of the areas with low pollen levels.
- **Aggregation:** Putting of similar sentences together to improve understanding and readability. For instance, merging the two sentences Grass pollen levels for Friday have increased from the moderate to high levels of yesterday and Grass pollen levels will be around 6 to 7 across most parts of the country into the single sentence Grass pollen levels for Friday have increased from the moderate to high levels of yesterday with values of around 6 to 7 across most parts of the country.
- **Lexical choice:** Using appropriate words that convey the meaning clearly. For example, deciding whether medium or moderate should be used when describing a pollen level of 4.
- **Referring expression generation:** Creating such referral expressions that help in identification of a particular object and region. For example, deciding to use in the Northern Isles and far northeast of mainland Scotland to refer to a certain region in Scotland. This task also includes making decisions about pronouns and other types of anaphora.
- **Realisation:** Creating and optimizing the text that should be correct as per the rules of grammar. For example, using will be for the future tense of to be.

NATURAL LANGUAGE GENERATION (NLG).

Machine learning systems can be differentiated into two types: Discriminative and Generative. While discriminative systems like classification, regression, clustering are the more well known type, it's the Generative systems that hold greater promise of achieving Artificial General Intelligence. In essence, a Generative system is expected to produce images, text or audio that would

be meaningful to the users. Generating a picture of a horse is a much harder problem than just identifying whether there is a horse in the picture.

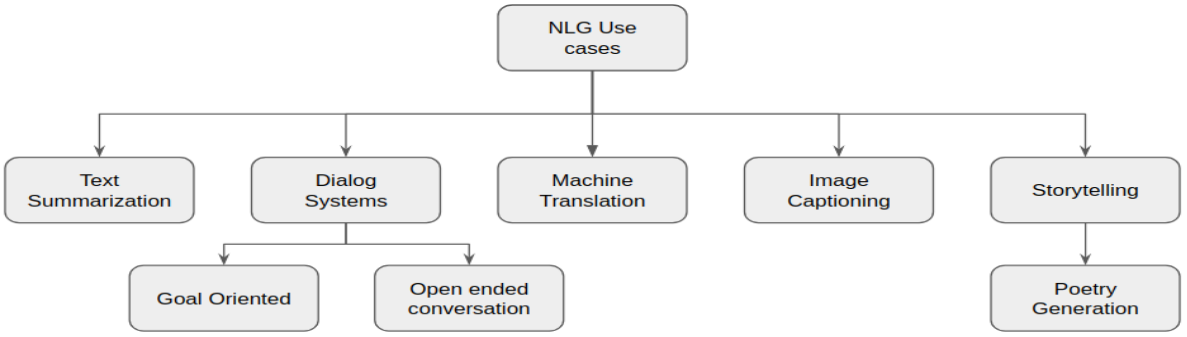
In this chapter, I will tackle the generative processes in NLP. Understandably, the field is called Natural Language Generation (NLG).

DEFINITION AND TAXONOMY

Reiter and Dale (2000) defined Natural Language Generation (NLG) as "the sub-field of artificial intelligence and computational linguistics that is concerned with the construction of computer systems than can produce understandable texts in English or other human languages from some underlying non-linguistic representation of information".

Two aspects need to be highlighted. First is generation of *understandable* text in a human language and second is the input to such a generation system is *non-linguistic* representation of information. For our purposes, we will drop the second requirement; which means that the source can be text as well. With such a definition in mind, we can have following taxonomy of NLG systems :

1. Text-to-Text
    a. Machine Translation : Automatically translating between various human languages
    b. Text Summarization : Summarizing a (big) text document into a shorter summary/abstract.
2. Data-to-Text
    a. Image Captioning : Describe the image in a short sentence.
    b. Video Captioning : Generate captions that change with the video scenes.
    c. Business Intelligence : Creating text summaries of data from conventional databases (e.g SQL)
3. Ideas-to-Text
    a. Poetry/Song Generation : Generating a song from a few keywords or mimicking the style of a certain artist.
    b. Fake News : Automatically generating news items that look credible but are not.
4. Dialog Systems (Chatbots)
    a. Goal Oriented : Chatting with a computer system (agent) with a specific purpose (e.g. booking a flight)
    b. Open ended conversations : When the conversation with the agent is casual chit-chat but has the components of information, emotion and human like empathy.



UNIT 4

Natural Language Generation: Natural Language Generation (NLG):

LECTURE NO:03

NLG Use Cases

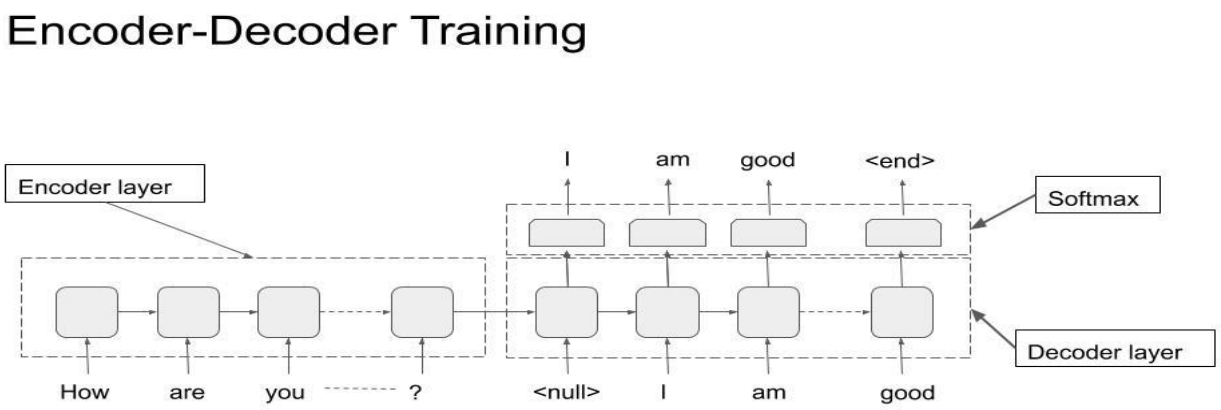
COMMON ARCHITECTURES

There are many architectures that are common across NLG systems. While some are used in other NLP domains as well, in the following sections I will explain them with a focus on language generation.

ENCODER-DECODER ARCHITECTURE

The most ubiquitous architecture for NLG is the encoder-decoder architecture, and especially the decoder part of it. Hence I will explain it in some detail. The architecture is shown in the subsequent figures.

The architecture can be seen as modeling conditional probability P(y/x)�(�/�) with y� being the output of the decoder and it is conditioned on x� (the output of the encoder). Hence the NLG task becomes generating text through decoder conditioned on some input, coming from the encoder.

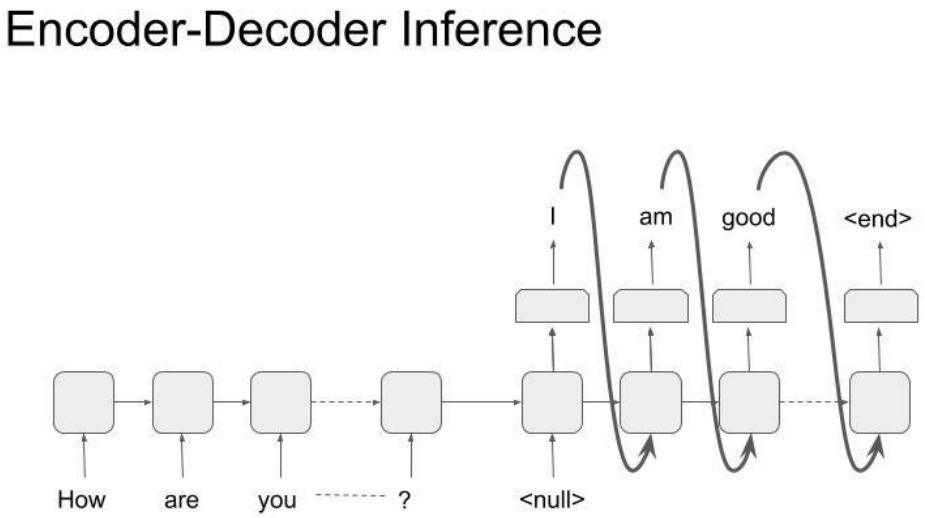


Encoder-Decoder (Training)

ENCODER :

As stated above, the purpose of this part of the network is to provide conditional information on which the decoder generates text. As such, this part can have **ANY** architecture that provides some form of embedding of the input. It can be a convolutional neural network to condition the generated text on some properties of an image (for example image captioning), or RNN/LSTM/Transformer architecture for text or audio based conditioning; or even a simple feed forward network to condition it on relational database for example. For the purpose of illustration we will be using an RNN/LSTM with text as input condition (as shown in the figure).

The thing to note here is that the richer the feature vector going from encoder to decoder, the more information decoder would have to generate output. This was the motivation to move from single feature vector to multiple vectors and to attention based models. This trend finally led to the transformer based models.



Encoder-Decoder (Inference)

DECODER :

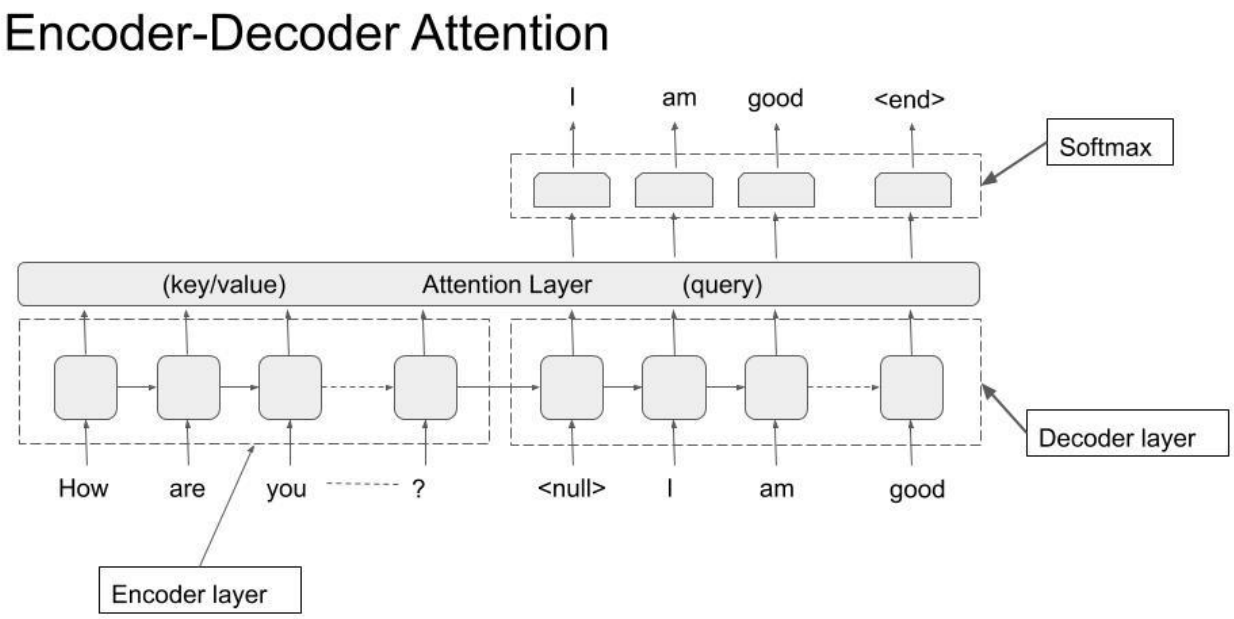
The decoder is the most distinctive part of an NLG system. Almost all decoders have the same form as shown in the figures above. The purpose is to generate text tokens one after the other until a terminating criteria is met. This termination is usually a termination token (<end> in the figures) or a max length criteria.

During training, we are given an input (text/image/audio) and the 'gold label text' that we want the system to learn to generate for that particular input. In the given example, the input is the text "How are you?" and the gold label is "I am good". The input goes through the encoder and produces a feature vector that is used as the input to decoder. The decoder then generates tokens one by one and the loss is calculated after the softmax layer from the generated token and the gold label token. Note the inclusion of an extra token '<null>' as the first token. The last token of the gold label should produce the '<end>' token.

During inference, we don't have the gold label, so the output of one step is used as input to next step, as shown in the figure. Note that it matches with the setup during training. The generator stops when '<end>' token is emitted; thus completing the inference.

13.3.2 Attention Architecture

The attention architecture is introduced in detail in Chapter 9. Here I will briefly mention its use in NLG systems. Looking at the picture below, we can see that the attention is from decoder to encoder.



ENCODER-DECODER (ATTENTION)

In other words, before generating each token, the decoder attends to all tokens in the encoder. The query is the decoder token and key/values are all encoder token. That way the decoder has much richer information to base its output on.

DECODING ALGORITHM AT INFERENCE

Now I will explain the decoding algorithms that are used to generate text from the softmax layer.

As explained above, during inference, the tokens are generated sequentially. In a vanilla version of decoding, at each step of the sequence, the token with highest probability in the softmax layer is generated. This is called 'greedy decoding', but it has been shown to produce suboptimal text. There are few improvements over this greedy approach.

BEAM SEARCH

In greedy decoder we simply output the maximum probability token at each step. But if we track multiple words at each step and then output the sequence formed by highest probability combination, we get beam search. The number of tokens we keep track of is the length of beam (k). The algorithm then goes as follows:

1. Select k-tokens with highest probability at step 1.
2. Use these k-tokens to generate k softmax vectors at step 2.
3. Keep the k highest scoring combinations.

4. Repeat steps 2 and 3 till <end> token is generated, or a predefined max is reached
5. At each step, we have only k-hypothesis, which is the length of beam search.

While beam search tends to improve the quality of generated output, it has its own issues. Chiefly among them is that it tends to produce shorter sequences. Although it can be controlled by the max parameter (of step 4), it's another hyperparameter to be reckoned with.

PURE SAMPLING DECODER

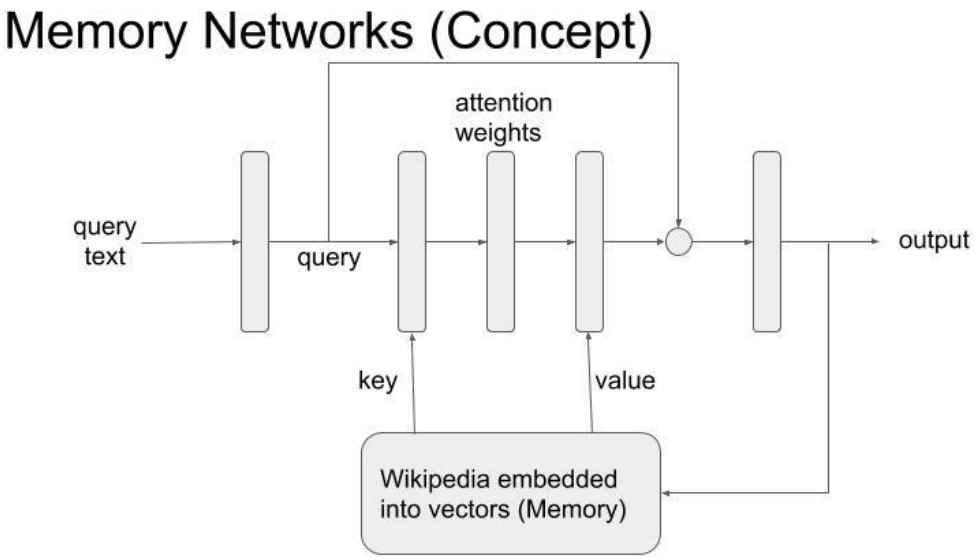
Here, at each step, instead of taking the token with maximum probability like in greedy search, the token is sampled from the whole vocabulary according to the probability distribution predicted by the softmax layer.

K-SAMPLING DECODER

It's like pure sampling decoder, but instead of sampling from whole vocabulary, the token is sampled only from the k-highest probability tokens.

MEMORY NETWORKS

Memory Networks is another architecture that is potentially quite useful in language generation tasks. The basic premise is that LSTMs/RNNs and even Transformer architecture stores all the information only in the weights of the network. When we want to generate text that should include information from a large knowledge base, this 'storage' of network weights is insufficient. Memory networks resolve this problem by employing an external storage (the memory) that it can use during language generation. Conceptual diagram is showing in the following figure, followed by a brief description.



Encoder-Decoder (Attention)

Memory (M) in this context can be any database that can be queried by the network. Usually it is of the form of key-value pairs or a simple array of vectors embedding a corpus of knowledge (eg DBPedia/wikipedia). For any query input (x), first we get an embedding. This is then used to attend over the memory M in the usual attention mechanism (Chapter 9). The output is the weighted sum of memory that incorporates information from complete knowledge corpus. In some cases the output can also be used to update the memory database.

LANGUAGE MODELS

Language models are probably the most important ingredient of generating text. As the name implies, they model the probability distribution of generating words and characters. More concretely, they model the conditional probability distribution P(w_t/w_{t-1}). Thus with a given input vector coming from a source (image/database/text), this model can be used to generate words one after the other.

UNIT 4

Natural Language Generation: Natural Language Generation (NLG):

LECTURE NO:04

DIALOG SYSTEMS

These are the systems where an agent chats with a human being either with a specific purpose (goal oriented) or it is a general open ended chat. The examples of goal oriented chats include tasks like booking an appointment or a flight ticket. Open ended chats can be talking about a general topic which may or may not include a ‘personality’ for the chatbot.

TYPES

- Chatbots
  - Open domain
  - Goal Oriented

ARCHITECTURES

Chatbots are complex software systems involving many modules and components that interact with each other asynchronously and in an event-driven fashion. An example architecture is shown in the following figure. This is an example of a goal-oriented chatbot which helps the use make a flight reservation. Following could be an example sequence of steps

1. The first step is to decode the text coming from a user. The two types of decoders that are usually required are an 'intent classifier' and a 'Named Entity Recognizer (NER)'. The intent classifier tries to determine the intent of the command coming from the user. So for the given example the command is 'Book a flight to Berlin on Tuesday', the correct classification would be 'book-flight'.

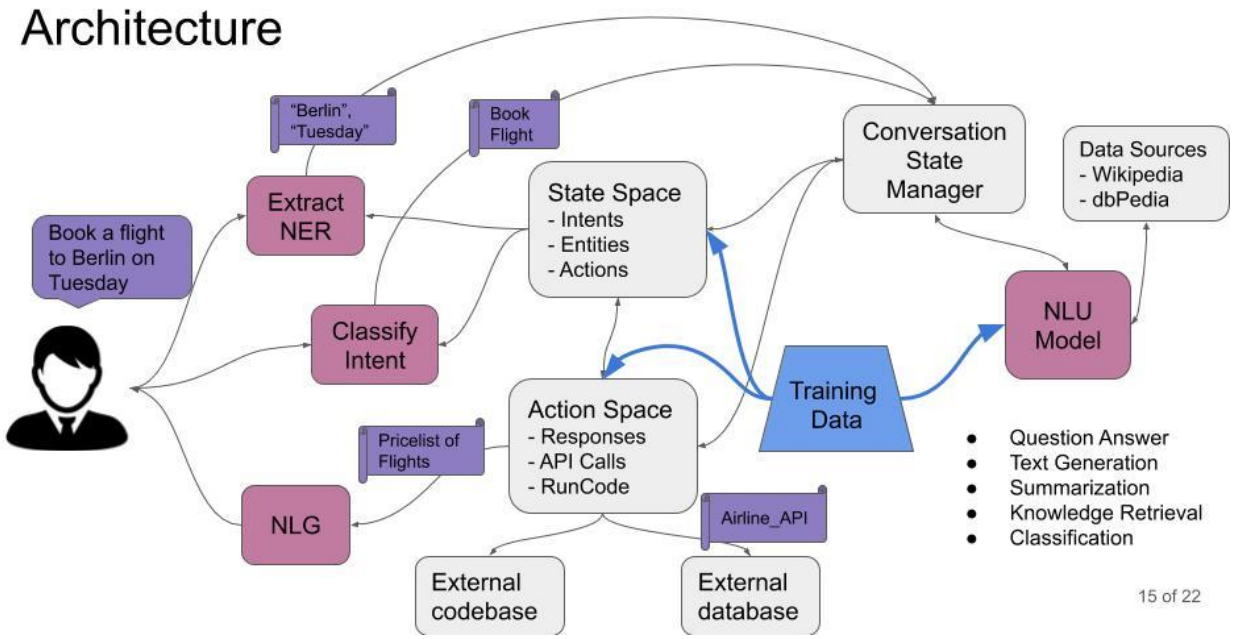
2. This intent is then fed to a 'State Space' module that keeps a list of entities that are required for each intent. For 'book-flight' intent, it would need entities like destination, date of flight, time of flight, airline/cost preferences etc. The NER in step 1 tries to extract these entities from the command. In our example, it could extract the destination and date of travel.

3. The conversation state manager keeps track of which entities have been resolved and which are outstanding. For both resolved and outstanding entities it refers to 'Action Space' module that generates API calls to external databases for requested parameters. It also refers to a 'Natural Language Understanding (NLU) model' to extract the information it needs to resolve the entities.

4. The 'Action Space' module calls various airline APIs and databases for associated tasks like determine available flights, prices and timings etc. Based on this information it refers back to the user to ask for unresolved entities.

5. The cycle continues till the system has all the required information to carry out the task.

A chatbot touches upon all areas of NLP and NLG; for example in the above case, it would need QnA ability, text generation, summarization, information retrieval and classification, to name a few.



Chatbot Architecture



Image Captioning Examples

IMAGE CAPTIONING SYSTEM

As the name suggests, these systems attempt to capture the caption of a given image as shown in the following figure. The input to the system is an image which is reduced to a feature vector using an appropriate architecture. Usually convolution based pretrained architectures are employed at this stege. The feature vector is then fed to a decoder network that produces the caption of the image. (See figure 2).

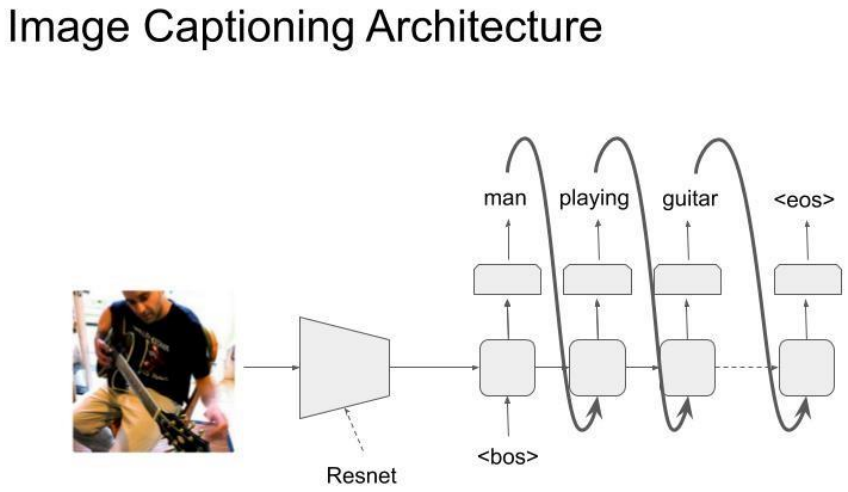


Image Captioning Concept

EXPERIMENTS

For this seminar, I implemented a network for this task. For the encoder part, I used a pretrained Resnet backbone with a trainable fully connected layer appended after that. This gave me a feature vector for an image. For decoder part, I used three different kind of decoder architectures.

- Simple LSTM Layer
- BERT decoder
- GPT-2 decoder

However, these decoders expect a multi-token input feature embedding on which attention can be implemented. But I have only one feature vector, so it cannot be used as-is in the decoder network. Therefore I came up with a different architecture in which this feature vector is added to the word and position embeddings of each output token (as shown in figure 3).

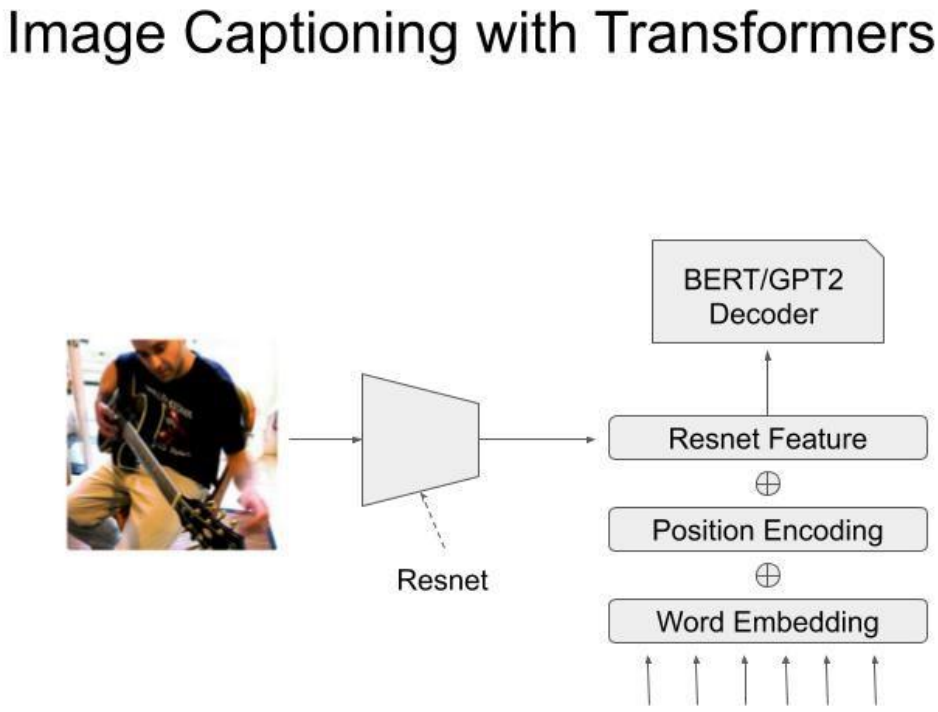


Image Captioning Architecture

13.5.2 Implementation

The network was implemented in PyTorch, using the Pytorch-Lightning and Nepture.AI frameworks for experimentation. I used MS-COCO dataset (2017) which had image captioning task. The encoder and LSTM decoder code is appended below for reference, while the BERT and GPT2 decoders were used from the HuggingFace library.

```
class EncoderCNN(nn.Module):
    def __init__(self, embed_size):
        """Load the pretrained ResNet-152 and replace top fc layer."""
        super(EncoderCNN, self).__init__()
        resnet = models.resnet152(pretrained=True)
        modules = list(resnet.children())[:-1]      # delete the last fc layer.
        self.resnet = nn.Sequential(*modules)
        self.linear = nn.Linear(resnet.fc.in_features, embed_size)
        self.bn = nn.BatchNorm1d(embed_size, momentum=0.01)

    def forward(self, images):
        """Extract feature vectors from input images."""
        with torch.no_grad():
            features = self.resnet(images)
        features = features.reshape(features.size(0), -1)
        features = self.bn(self.linear(features))
        return features
class DecoderRNN(nn.Module):
    def __init__(self, embed_size, hidden_size, vocab_size, num_layers, max_seq_length=20):
        """Set the hyper-parameters and build the layers."""
        super(DecoderRNN, self).__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, num_layers, batch_first=True)
        self.linear = nn.Linear(hidden_size, vocab_size)
        self.max_seg_length = max_seq_length

    def forward(self, features, captions, lengths):
        """Decode image feature vectors and generates captions."""
        embeddings = self.embed(captions)
        embeddings = torch.cat((features.unsqueeze(1), embeddings), 1)
        packed = pack_padded_sequence(embeddings, lengths, batch_first=True)
        hiddens, _ = self.lstm(packed)
        outputs = self.linear(hiddens[0])
        return outputs

    def sample(self, features, states=None):
        """Generate captions for given image features using greedy search."""
        sampled_ids = []
        inputs = features.unsqueeze(1)
        for i in range(self.max_seg_length):
            hiddens, states = self.lstm(inputs, states) # hiddens: (batch_size, 1, hidden_size)
            outputs = self.linear(hiddens.squeeze(1))   # outputs:  (batch_size, vocab_size)
            _, predicted = outputs.max(1)               # predicted: (batch_size)
            sampled_ids.append(predicted)
            inputs = self.embed(predicted)              # inputs: (batch_size, embed_size)
            inputs = inputs.unsqueeze(1)                # inputs: (batch_size, 1, embed_size)
        sampled_ids = torch.stack(sampled_ids, 1)   # sampled_ids: (batch_size, max_seq_length)
        return sampled_ids
```

13.5.3 Results

For evaluation, I used perplexity on a held out validation set. The results for the three architectures are shown in the following figure. It can be seen that BERT outperforms LSTM with a lower perplexity and GPT2 outperforms both LSTM and BERT model.



LSTM BERT GPT2

Image Captioning Results

UNIT 4

Natural Language Generation: Natural Language Generation (NLG):

LECTURE NO: 05

APPLICATION OF NLG.

Techniques for Evaluating NLG systems

1. **Task-based evaluation:** It includes human-based evaluation, who assess how well it helps him perform a task. For example, a system which generates summaries of medical data can be evaluated by giving these summaries to doctors and assessing whether the summaries help doctors make better decisions.
2. **Human ratings:** It assess the generated text on the basis of ratings given by a person on the quality and usefulness of the text.
3. **Metrics:** It compares generated texts to texts written by professionals

An example of an interactive use of natural language generation is the **WYSIWYM framework,** which stands for **"*What you see is what you meant* "** It allows users to see and manipulate the continuously rendered view (NLG output) of an underlying formal language document (NLG input), thereby editing the formal language without learning it.
Another example includes **Content generation systems** that assist human writers and makes the writing process more efficient and effective. A content generation tool based on web mining using search engines APIs has been built. The tool imitates the cut-and-paste writing scenario where a writer forms its content from various search results.
So far, the most successful NLG applications have been **Data-to-Text systems**, which generate textual summaries of databases and data sets; these systems usually perform data analysis as well as text generation. In particular, several systems have been built that produce textual weather forecasts from weather data.

APPLICATIONS OF NATURAL LANGUAGE GENERATION

Natural language generation, or NLG, has numerous important uses in a range of sectors. The following are a few notable domains where NLG is extensively employed

INTELLIGENT AUTOMATION AND REPORTING:

- NLG is used to transform analytical and complex data into reports and summaries that are understandable to humans. This makes it especially easy for stakeholders to comprehend and act upon insights in business intelligence.

MARKETING AND CONTENT CREATION:

- NLG is used to create content for blogs, websites, and advertising collateral. It can produce written materials at scale, including product descriptions and promotional content.

Virtual assistants and chatbots:

- By allowing chatbots and virtual assistants to respond in natural language, natural language generation (NLG) improves their conversational skills. Ensuring a user experience that is both engaging and human-like is imperative.

Analysis of Finance and Investments:

- Using numerical data and trends, natural language generation (NGL) is used in the finance industry to automatically produce financial reports, investment summaries, and market commentary.

Medical Records:

- NLG is used to generate medical reports, documentation, and patient summaries from electronic health records (EHR). In medical settings, it can simplify the documentation procedure.

Educational Content and E-Learning:

- NLG contributes to the creation of instructional materials, assessments, and personalised feedback for students. It aids in the creation of learning platforms that are adaptable.

WHAT ARE SOME COMMON EXAMPLES OF NATURAL LANGUAGE GENERATION (NLG)?

The following are some of the several real-life applications of natural language generation (NLG):

- **Chatbots:** AI-powered NLG allows chatbots to have more engaging and natural conversations with users, understanding their intent and responding in a relevant and personalised way.
- **News & Report Generation:** NLG can automatically generate news articles or reports based on data, saving time and resources for journalists and analysts.
- **Personalised Marketing:** AI-powered NLG can create personalised marketing messages that resonate with individual customers, leading to improved engagement and conversion rates.
- **Accessibility Tools:** NLG can be used to convert complex information into simpler formats, making it accessible to people with disabilities.

UNIT 4

Machine Translation

LECTURE NO: 05

MACHINE TRANSLATION APPROACHES AND SURVEY FOR INDIAN LANGUAGES

The term Machine Translation is a standard name for computerized systems responsible for the production of translations from one natural language into another with or without human assistance. It is a sub-field of computational linguistics that investigates the use of computer software to translate text or speech from one natural language to another. Many attempts are being made all over the world to develop machine translation systems for various languages using rule-based as well as statistically based approaches. Development of a full-fledged bilingual machine translation (MT) system for any two natural languages with limited electronic resources and tools is a challenging and demanding task. In order to achieve reasonable translation quality in open source tasks, corpus based machine translation approaches require large amounts of parallel corpora that are not always available, especially for less resourced language pairs. On the other hand, the rule-based machine translation process is extremely time consuming, difficult, and fails to analyze accurately a large corpus of unrestricted text. Even though there has been effort towards building English to Indian language and Indian language to Indian language translation system, unfortunately, we do not have an efficient translation system as of today. The literature shows that there have been many attempts in MT for English to Indian languages and Indian languages to Indian languages. At present, a number of government and private sector projects are working towards developing a full-fledged MT for Indian languages. This paper gives a brief description of the various approaches and major machine translation developments in India.

Introduction MT refers to the use of computers to automate some of the tasks or the entire task of translating between human languages. Development of a full-fledged bilingual MT system for any two natural languages with limited electronic resources and tools is a challenging and demanding task. Many attempts are being made all over the world to develop MT systems for various languages using rule-

based as well as statistical-based approaches. MT systems can be designed either specifically for two particular languages, called a bilingual system, or for more than a single pair of languages, called a multilingual system. A bilingual system may be either unidirectional, from one Source Language (SL) into one Target Language (TL), or may be bidirectional. Multilingual systems are usually designed to be bidirectional, but most bilingual systems are unidirectional. MT methodologies are commonly categorized as direct, transfer, and Interlingua. The methodologies differ in the depth of analysis of the SL and the extent to which they attempt to reach a language independent representation of meaning or intent between the source and target languages. Barriers in good quality MT output can be attributed to ambiguity in natural languages. Ambiguity can be classified into two types: structural ambiguity and lexical ambiguity. India is a linguistically rich area. It has 18 constitutional languages, which are written in 10 different scripts. Hindi is the official language of the Union. Many of the states have their own regional language, which is either Hindi or one of the other constitutional languages. In addition, English is very widely used for media, commerce, science and technology, and education only about 5% of the world's population speaks English as a first language. In such a situation, there is a large market for translation between English and the various Indian languages. Even though MT in India started more than two decades ago, it is still an ongoing process. The third section of this paper discusses various approaches used in English to Indian languages and Indian language to Indian language MT systems. The fourth section gives a brief explanation of different MT attempts for English to Indian languages and Indian languages to Indian languages. 2. History of MT The major changeovers in MT systems are as shown in Figure 1. The theory of MT pre-dates computers, with philosophers 'Leibniz and Descartes' ideas of using code to relate words between languages in the seventeenth century (Hutchins et al., 1993). The early 1930s saw the first patents for 'translating machines'. Georges Artsrouni was issued a patent in France in July 1933. He developed a device, which he called a 'cerveau mécanique' (mechanical brain) that could translate between languages using four components: memory, a keyboard for input,

MT APPROACHES

Generally, MT is classified into seven broad categories: rule-based, statistical-based, hybrid-based, example-based, knowledge-based, principle-based, and online interactive based methods. The first three MT approaches are the most widely used and earliest methods. Literature shows that there have been fruitful attempts using all these approaches for the development of English to Indian languages as well as Indian languages to Indian languages. At present, most of the MT related research is based on statistical and example-based approaches. Figure 2 shows the classification of MT in Natural language Processing (NLP).

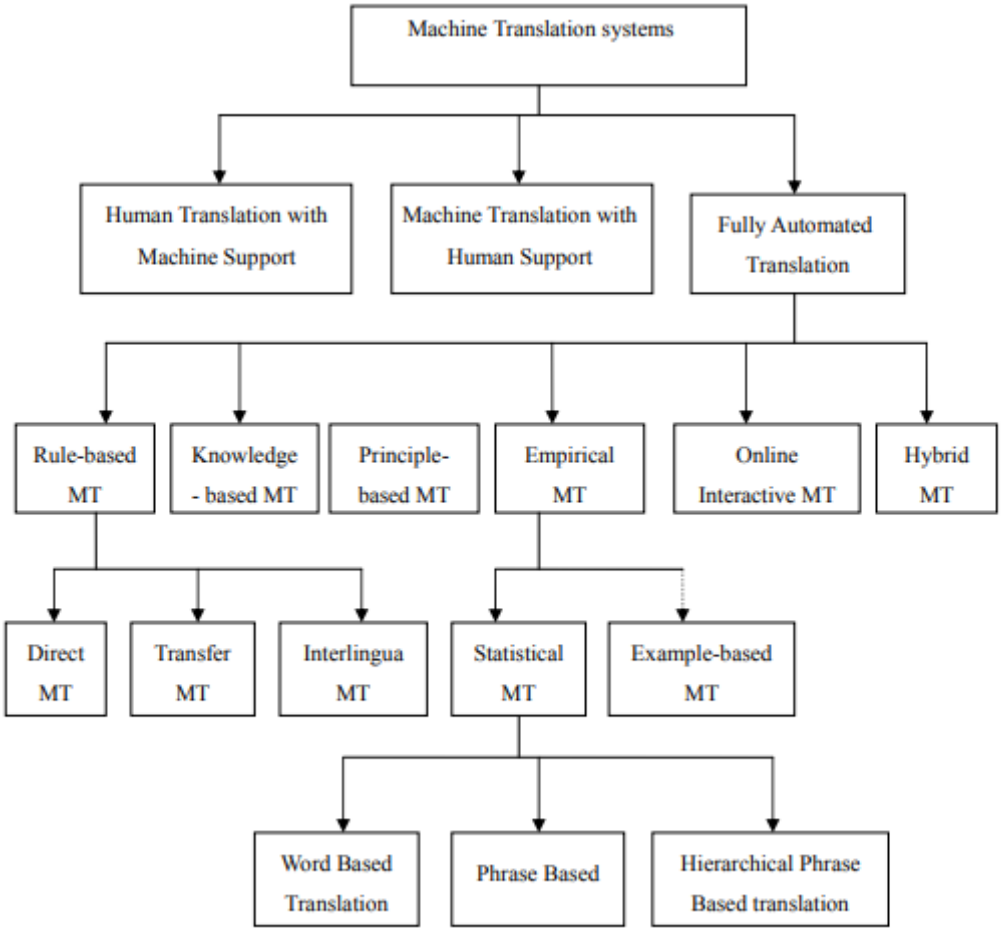


Figure 2. Classification of MT System.

UNIT 4

Machine Translation

LECTURE NO: 06

OVERVIEW

Machine Translation (MT) is a domain of computational linguistics that uses computer programs to translate text or speech from one language to another with no human involvement with the goal of relatively high accuracy, low errors, and effective cost.

Machine Translation is a very important yet complex process as there is currently a huge number of nuanced natural languages in the world.

WHAT IS A MACHINE TRANSLATION, AND HOW DOES IT WORK?

Need for Machine Translation

There is an explosion of data in the modern world with the information revolution and since language is the most effective medium of communication for humans, there is also an increased need for translating from one language to another using NLP tools and other techniques.

- The demand for translation is majorly due to the exponential rise increase in the exchange of information between various regions using different regional languages.
- **Examples** such as access to web documents in non-native languages, using products from across countries, real-time chat, legal literature, etc. are some use cases.

WHY IS MACHINE TRANSLATION SO HARD TO TACKLE?

Machine translation using NLP tools from one language to another is notoriously difficult due to the presence of many words with multiple meanings, sentences with potential possible readings, and the fact that certain grammatical relations in one language might not exist in another language.

PROCESS OF MACHINE TRANSLATION

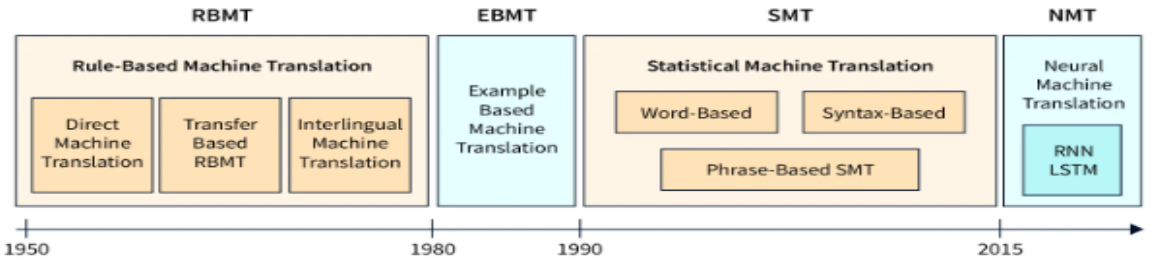
The basic requirement in the complex cognitive process of machine translation is to understand the meaning of a text in the original (source) language and then restore it to the target (sink) language.

The primary steps in the machine translation process are:

- We need to decode the meaning of the source text in its entirety. There is a need to interpret and analyze all the features of the text available in the corpus.
- We also require an in-depth knowledge of the grammar, semantics, syntax, idioms, etc. of the source language for this process.

- We then need to re-encode this meaning in the target language, which also needs the same in-depth knowledge as the source language to replicate the meaning in the target language.

EVOLUTION OF MACHINE TRANSLATION



- 1950 to 1980: The major interest and support for machine translation in the decades from the 1950s to the 1980s was fueled by the need for high-speed, high-quality translation of arbitrary texts, which are heavily funded mostly by military and intelligence communities during the cold war era.
    - The systems during this time were mainly rule-based, and almost all research and development centred around improving the performance of individual use cases.
- 1980-1990: In the late 1980s, example-based machine translation models were employed for some time till the early 1990s.
    - The main idea is to provide the system with a set of sentences within the source language and then the corresponding translations in the target language. Both examples serve as examples for translations.
- 1990-2015: Machine translation systems based on statistical methods were introduced in the 1990s, replacing the earlier rule-based and example-based systems.
    - The statistical-based methods come under the case of corpus-based machine translation methods.
- 2015-Present: Deep neural network-based models were developed for automatic machine translation by almost all technology companies for industrial applications, which achieved state-of-the-art results in the last few years recently among all NLP techniques.
    - This recent utility of neural net-based models made the field named neural machine translation.

DIFFERENT TYPES OF MACHINE TRANSLATION IN NLP

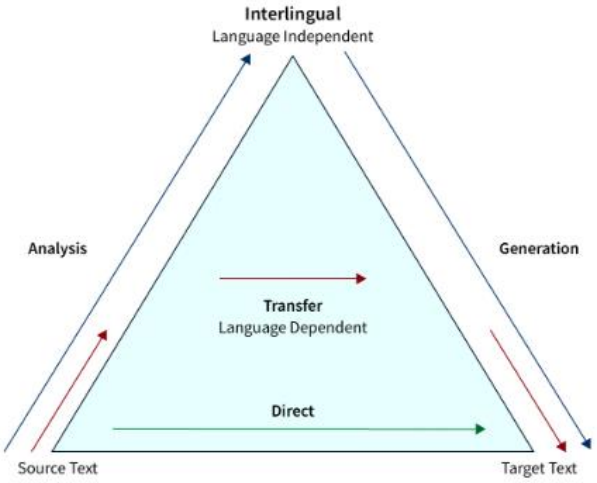
Machine translation methods are mainly classified into **rule-based and corpus-based methods** during the developments from the 1950s till now. Recently neural network-based models are primarily employed for translation tasks using NLP.

Let us discuss and learn the classification and methods involved in each type of model.

RULE-BASED MACHINE TRANSLATION OR RBMT

Also called **knowledge-based machine translation**, these are the earliest set of classical methods used for machine translation.

- These translation systems are mainly based on **linguistic information** about the source and target languages that are derived from dictionaries and grammar covering the characteristic elements of each language separately.
- Once we have input sentences in some source languages, RBMT systems generally generate the translation to output language based on the morphological, syntactic, and semantic analysis of both the source and the target languages involved in the translation tasks.
- The sub-approaches under RBMT systems are the direct machine translation approach, the interlingual machine translation approach, and the transfer-based machine translation approach.



CORPUS-BASED MACHINE TRANSLATION APPROACH OR CBMT

Also called data **driven machine translation**, these methods majorly overcome the problem of knowledge acquisition problem of rule-based machine translation approaches. The Corpus-based machine translation methods can handle linguistic issues and generally find it difficult to translate cultural items and concepts.

- Corpus-Based Machine Translation is a set of methods that uses a **bilingual parallel corpus** to obtain knowledge for new incoming translation.
- This approach uses a large amount of raw data in the form of parallel corpora. This raw data contains text and their translations. These corpora are used for acquiring translation knowledge.
- Corpus-based approaches are classified into **two sub-approaches**: Statistical Machine Translation and the Example-based Machine Translation approach.

EXAMPLE-BASED MACHINE TRANSLATION OR EBMT

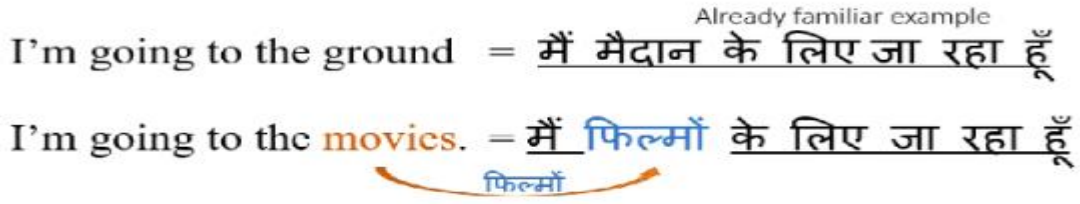
Example-based machine translation systems are trained from **bilingual parallel corpora** where the sentence pairs contain sentences in one language with their translations into another.

- The foundational idea of example-based machine translation is the concept of translation by analogy.
    - The principle of translation by analogy is encoded to the example-based machine translation through the example translations that are used to train the system.

- The bilingual corpus with parallel texts serves as the main knowledge for an Example-based machine translation system in which translation by analogy.
    - A set of sentences in the source language are given in the example-based machine translation system, and corresponding translations of each sentence in the target language with point-to-point mapping are also provided as input.
    - These examples are used to translate similar types of sentences of the source language to the target language.
- Example acquisition, example base, and management, example application, and synthesis are the four main tasks to be performed in EBMT methods.

SAMPLE EXAMPLE FOR EBMT-BASED APPROACH:

- Let us say we want to translate a simple sentence — I'm going to the playground., and let's say that we have already translated another similar sentence — I'm going to the movies. — and we can find the word "movies" in the dictionary.
- All we need is to figure out the difference between the two sentences, and then translate the missing word in the new language.
- The more examples we have, the better the translation will be.



Example-based machine translation systems paved the path further to statistical-based machine translation systems.

STATISTICAL MACHINE TRANSLATION OR SMT

Statistical machine translation (SMT) models are generated with the use of statistical models whose parameters are derived from the analysis of bilingual text corpora. The ideas for SMT come from information theory.

- The principle theory of SMT-based models is that they rely on the **Bayes Theorem** to perform the analysis of parallel texts.
    - The central idea is that every sentence in one language is a **possible translation** of any sentence in the other language, but the most appropriate is the translation that is assigned the highest probability by the system.
- The SMT methods need millions of sentences in both the source and target languages for analysis to collect the relevant statistics for each word.
    - In the early set of models, the abstracts of the European Parliament and the `United Nations Security Council meetings, which were available in the languages of all member countries, were taken for getting the required probabilities.
- SMT-based methods are further **classified** into Word-based SMT, Phrase-based SMT, and Syntax based SMT approaches.



ISSUES WITH SMT-BASED APPROACHES

- **Sentence Alignment**: As we are using two corpora to match sentences, many times we will encounter single sentences in the source language, which can be found translated into several sentences in the target language.
  - **Example:** When translating from English to German, the sentence "Raghu does not live here," the word "does" doesn't have a clear alignment in the translated sentence "Raghu wohnt hier nicht."
- **Statistical Anomalies**: Real-world training sets may override translations like proper nouns.
  - **Example:** "I took the train to Berlin" gets mistranslated as "I took the train to Paris" due to an abundance of "train to Paris" in the training set generally.
- **Idioms**: Depending on the corpora used, idioms may not translate idiomatically into the target language.
  - **, For example,**, using Canadian Hansard as the bilingual corpus, "hear" may almost invariably be translated to "Bravo!" since in Parliament, "Hear, Hear!" becomes "Bravo!".
- **Different Word Orders**: Since the word orders in languages differ, Statistical machine translation methods do not work well between languages that have significantly different word orders, **for example** Japanese and European languages.
- **Corpus Creation**: This can be very costly for users with limited resources, and sometimes the results are also unexpected where the superficial fluency can be deceiving.
- **Data Dilution**: This is one of the most common anomalies caused when attempting to construct a new statistical model to represent a distinct terminology for a specific corporate brand or domain.

HYBRID MACHINE TRANSLATION OR HMT

- Hybrid Machine Translation approaches take advantage of both statistical and rule-based translation methodologies which were proven to have better efficiency in the area of machine translation systems using NLP.
  - These techniques are better than the previous methods and have more power, flexibility, and control in translation tasks in general.
  - Most governmental and industrial-based machine translation systems use the hybrid-based approach to develop translation from source to target language, which is based on both rules and statistics.
- Hybrid-based approaches combine the best of both worlds such that they can be used in several different ways.
  - In cases where translations are performed in the first stage using a rule-based approach, they can be followed by adjusting or correcting the output using statistical information.
  - On the other way, rules are used to pre-process the input data as well as post-process the statistical output of a statistical-based translation system.

NEURAL MACHINE TRANSLATION OR NMT

Neural Machine Translation methods in NLP employ artificial intelligence techniques and, in particular, rely upon neural network models which were originally based on the human brain to build machine learning models with the end goal of translation and also further learning languages, and improving the performance of the machine translation systems constantly.

- Neural Machine Translation systems do not need any specific requirements that are regular to other machine translation systems like statistical-based methods in general.
- Neural Machine Translation works by incorporating training data, which can be generic or custom depending on the user's needs.
    - **Generic Data**: Includes the total of all the data learned from translations performed over time by the machine translation engines employed in the entire realm of translation.
        - This data also serves as a generalized translation tool for various applications, including text, voice, and documents.
    - **Custom or Specialized Data**: These are the sets of training data fed to machine translation tasks to build specialization in a subject matter.
        - Subjects include engineering, design, programming, or any discipline with its specialized glossaries and dictionaries.

Advantages of NMT Models:

- NMT has achieved the state of the art performance in large-scale translation tasks in many languages like English to French, English to German etc.
- NMT requires minimal domain knowledge and is conceptually very simple and powerful.
- NMT has a small memory footprint as it does not store gigantic phase tables and language models.
- NMT also can generalize well to very long word sentences.

UNIT 4

Machine Translation

LECTURE NO: 07

WHAT ARE THE BENEFITS OF MACHINE TRANSLATION?

Machine translation is a very important tool in the translation process of modern-day application workflows.

Let us look at the primary benefits offered by using MT approaches:

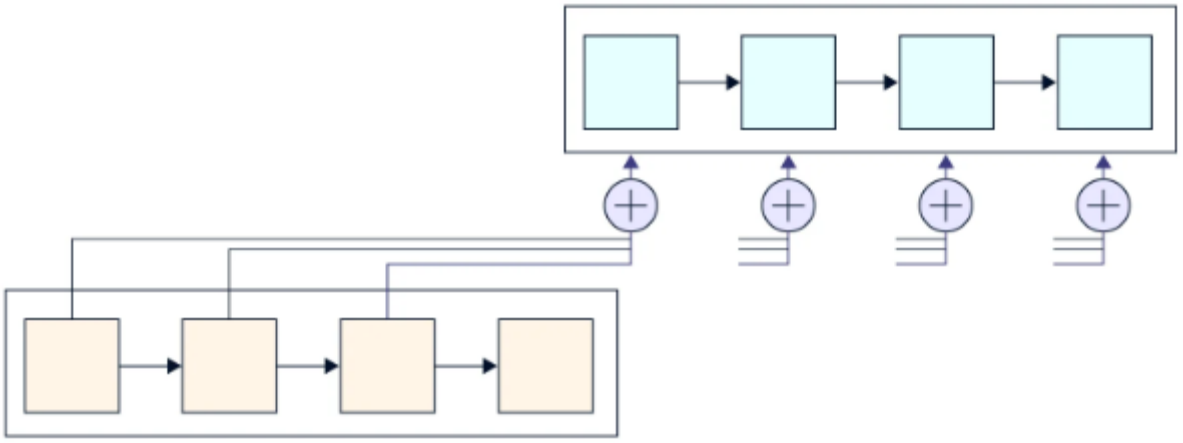
- **Fast Translation Speed**: Machine translation using NLP has the potential to translate millions of words for high-volume translations happening in modern day-to-day processes.
  - MT cuts the human translator effort usually done post-editing.
  - We can use machine translation to tag high-volume content and organize the contents so that we can look up over search or retrieve content in translated languages quickly.
- **Excellent Language Selection**: Most machine language translation tools these days can translate more than 100 languages.
  - The MT programs are powerful enough to translate multiple languages at once so they can be rolled out to a global user base across products and do documentation updates on the fly.
  - MT is well-suited to many language pairs these days (English to most European languages) with very high accuracies.
- **Reduced Costs**: As annotation with humans doesn't scale well both in terms of cost and speed, we can use machine translation using NLP to cut translation delivery times and costs.
  - We can also produce basic translations with little compute costs that can be further utilized by human translators so that they can further refine and edit from previous steps.

SEQUENCE TO SEQUENCE MODEL

With the advent of **recurrent network models** for a variety of complex tasks where the order of the task (like the sequence of words in a sentence) is important, Sequence to Sequence has become very popular for machine translation tasks in NLP.

Sequence to Sequence models primarily consists of an encoder and decoder model architecture and utilize a combination of recurrent units in their layers.

- The encoder and the decoder are two distinct individual components working hand in hand together to produce state-of-the-art results while performing different kinds of computations.
- The input sequences of words are taken as the input by the encoder, and the decoder component typically has the relevant architecture to produce the associated target sequences.
- In the machine translation task, the source language is passed as input to the encoder as a sequence of vectors, and then it is passed through a series of recurrent units whose end output is stored as the last hidden state as the encoder state representation.
- This hidden state is used as input for the decoder and passed through a series of recurrent again as a for loop, usually each output sequentially so that the output produced has the highest probability.



ATTENTION MODEL

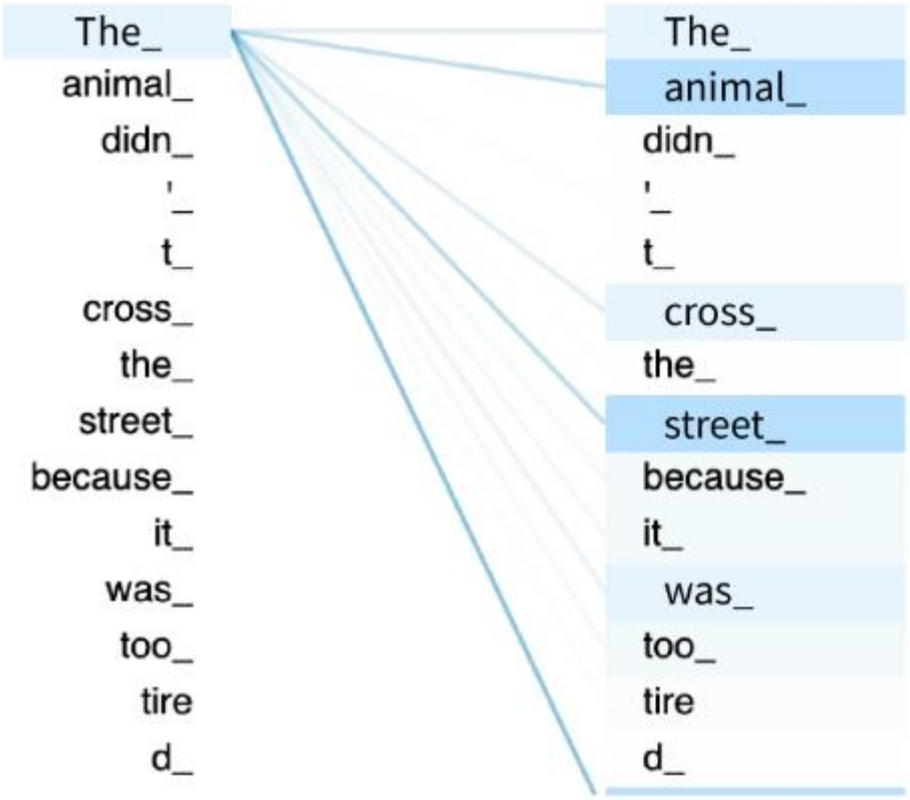
One of the potential issues with using encoder–decoder architecture of seq-2-seq models is that the neural network typically compresses all the necessary information of a source sentence into a fixed-length vector and also keeps track of what words are exactly important.

- The task is difficult for recurrent layers to cope with long sentences, especially those that are longer than the sentences in the input training corpus.
- Attention is proposed as a novel solution to the limitation of this particular encoder-decoder model of decoding long sentences by selectively focusing on sub-parts of the sentence during translation.

Intuition for Attention Models

Attention allows neural network models to approximate the visual attention mechanism humans use in processing different tasks.

- The initial step of the model works by studying a certain point of an image or text with intense high-resolution focus, like how humans process a new scene in an image or text while talking.
- While focussing on current high-resolution contexts, it also perceives the surrounding areas in low resolution simultaneously.
- The last step is to then adjust the focal point as the network begins to understand the overall picture together.



Using Attention in Neural Machine Translation Models

When using attention for neural machine translation tasks, the meaning of the whole sentence is mapped into a fixed-length vector, which then generates a translation based on that vector as a whole.

- The overall model translates the sentence by paying attention to the general, high-level overall sentiment of the sentence instead of only concentrating on doing word-to-word translation.
- Attention-based models for Neural Machine Translation are proven to drastically improve accuracy and are much easier to construct and faster to train the models.
- There are mainly two types of attention-based models, generalized/global attention and self-attention.

TRANSFORMER MODEL

Transformers models also come under sequence-to-sequence models for many computational tasks, but they rely mostly on the **attention mechanism** to draw the dependencies between layers of the network.

- When compared with the previous neural network architectures, the Transformer based neural attention models achieved the highest BLEU scores on machine translation tasks while also requiring significantly less compute and training time.
- The primary idea in the transformer is to apply a self-attention mechanism and model the relationships between all words in a sentence directly regardless of their respective position like in the recurrent networks.

Transformer also possesses the **encoder-decoder architecture** like in RNN-based Seq2Seq models for transforming one sequence into another.

- The main difference with the existing Seq2Seq models is that they do not imply any recurrent layers and instead handle the entire input sequence in at once and don't iterate word by word.
- The encoder is a combination of a Multi-Head Self-Attention Mechanism and a series of Fully-Connected Feed-Forward Networks.
- The decoder is a combination of Masked Multi-Head Self-Attention Mechanism, Multi-Head Self-Attention Mechanism, and also Fully-Connected Feed-forward Networks.

MACHINE TRANSLATION VS. HUMAN TRANSLATION

- While machine translation is the automatic and instant translation of text from one language to another using software tools, human translation, on the other hand, includes the actual brainpower of humans in the form of one or more translators translating the text manually.
- With the rise of AI technologies and open source APIs offered in the market, numerous translation tools are available at practically no or zero cost making it a reasonable answer for organizations who will be unable to manage the cost of expert translations.
- While machine translation is powerful to scale in terms of cost and speed, careful judgment needs to be exercised to call between human and machine translation which requires additional consideration and subtlety in the translation task.

APPLICATIONS OF MACHINE TRANSLATION

Let us discuss some of the major applications of machine translation employed in the industry currently.

Text Translation

Automated Text Translation is mainly used in an assortment of sentence-level and text-level translation applications.

- **Sentence-Level Translation** applications typically incorporate the translation of inquiry and recovery inputs and the translation of (**OCR**) outcomes of picture optical character acknowledgement.
- **Text-Level Translation** applications incorporate the translation of a wide range of unadulterated reports and the translation of archives with organized data.

Speech Translation

Voice input is ubiquitous in most modern applications with the exponential advancement of modern chips in mobile and processing voice input has come to be ingenious and necessary for human-computer utilization, and discourse translation has become a significant application in such situations.

- The typical cycle for discourse interpretation includes multiple steps like - source language discourse, then source language text, target language text, followed by target language discourse".
- The text translation step from source language text to target-language text is the module moderating machine translation task.

Other Applications

- Machine translation technology using NLP can help law firms and corporate legal departments understand and process large quantities of legal documents quickly.
- In construction and manufacturing projects, competitive bid documents translated from foreign languages can provide all the information needed so that one can act as quickly as possible and increase the chances of securing the contracts in respective fields.
- MT is also used in many other fields like e-commerce where content translation is done, and also in many web-related tasks.

CONCLUSION

- Machine translation is the process of automatic translation of text from one language to another using software tools.
- Machine translation tools evolved from using rule-based methods to corpus-based methods. Neural machine translation methods are the state of the models in MT tasks.
- Machine translation is widely used in text-based, speech-based, and many other industrial applications in modern mobile phones, web-based tasks, and most business settings.
- We will then compare machine translation with human translation and look at different state-of-the-art models for translation tasks.
- Machine translation tools using NLP can be employed to effectively scale across time and cost, but human translators with expert skills need to be employed in nuanced and creative translation tasks.
- Sequence to Sequence models employ encoder-decoder architecture and recurrent layers (like LSTM, RNN) to model machine translation tasks.
- Attentions are a set of novel computational mechanisms proposed to solve the problems encountered in recurrent mechanisms of Sequence to Sequence models.
- Transformers utilize attention mechanisms along with the encoder-decoder architecture to achieve state-of-the-art results in machine translation tasks.